

MATD10: Project 1 Report

Yuyang Wang

Chapter 1: Introduction

In this report, we present the results of our first project for the MATD10 course. The project focuses on investigating the order of convergence, stability, and the influence of CFL on the eight schemes mentioned in the course note. We will explain why our code works, and present the results of our numerical experiments, with a (hopefully) clear and detailed analysis on those results.

Chapter 2: Code Explanation

2.1 setup.py

The `setup.py` file contains the implementation of the functions that are used in the main code, such as `grid()`, `find_dt()`, `exact_sol()`, and `error()`. The first two functions are used to set up the grid spacing and time step for our numerical scheme, while the last three two are used to compute the exact solution and the error of our numerical solution.

The `grid()` function creates the grid of points $x_i = a + i \cdot \Delta x$, where $\Delta x = \frac{(b-a)}{nx}$, and $i = 0, 1, \dots, nx - 1$. It returns an array of points and the spacing between them. The `find_dt()` function computes the time step dt based on the spatial step dx , the wave speed c , and the CFL number.

The `exact_sol()` function computes the exact solution at a given time t based on the initial condition and the wave speed. Suppose our $xs = [x_0, x_1, \dots, x_{nx-1}]$ then

$$xs - ct = [x_0 - ct, x_1 - ct, \dots, x_{nx-1} - ct]$$

By definition:

$$xshift = a + \text{mod}(xs - ct - a, b - a)$$

Conceptually, if we want to wrap x outside of our solution interval back into it, we first $-a$ so

$$\text{mod}(x - a, b - a) \in [0, b - a)$$

and then $+a$ will shift it into $[a, b)$, as desired. Hence: $g(xshift)$ applies g to $xshift$ element-wise, which would lead to

$$[g(a + \text{mod}(x_0 - ct - a, b - a)), g(a + \text{mod}(x_1 - ct - a, b - a)), \dots, g(a + \text{mod}(x_{nx-1} - ct - a, b - a))]$$

The `error()` function computes the maximum pointwise error between the numerical solution and the exact solution. It takes in the numerical solution, the exact solution, and returns the maximum absolute difference between them. Specifically, it computes the error at each point as $error_i = u_{app}[i] - u_{exact}[i]$ and then returns $\max_i |error_i|$, which refers to the infinity-norm of the error.

2.2 schemes.py

The `schemes.py` file contains the implementation of the eight numerical schemes mentioned in the course note. Each scheme is implemented as a function that takes in the current solution and returns the next solution. The main code will call these functions iteratively to compute the solution at each time step.

Before delving into the details of each scheme, we will first convert the operators definitions into its corresponding equivalent “raw pieces”, which is my own choice of implementation.

$$\begin{aligned}
D_+ &= \frac{S_+ - I}{\Delta x} \\
D_- &= \frac{I - S_-}{\Delta x} \\
D_0 &= \frac{S_+ - S_-}{2\Delta x} = \frac{1}{2}(D_+ + D_-) \\
D_{xx} &= D_- D_+ = D_+ D_- = \frac{S_+ - 2I + S_-}{\Delta x^2}. \\
(S_+ U)_i &= U_{i+1} \\
(S_- U)_i &= U_{i-1}
\end{aligned}$$

2.2.1 Upwind

The upwind scheme in operator form is given by:

$$U^{j+1} = (I - c\Delta t D_{\pm})U^j$$

where D_+ is used when $c < 0$ and D_- is used when $c > 0$. A brief justification is that with $c < 0$, the wave is propagating to the left, so we should use the forward difference D_+ to approximate the spatial derivative. Conversely, with $c > 0$, the wave is propagating to the right, so we should use the backward difference D_- to approximate the spatial derivative. The component-wise form of the upwind scheme is given by:

$$u_i^{j+1} = \begin{cases} u_i^j - \frac{c\Delta t}{\Delta x}(u_i^j - u_{i-1}^j), & c > 0, \\ u_i^j - \frac{c\Delta t}{\Delta x}(u_{i+1}^j - u_i^j), & c < 0. \end{cases}$$

The corresponding scheme python code is very straightforward to implement, as we can directly use the definition of D_+ and D_- to compute the next solution, and also after recognizing `np.roll(u, -1)` and `np.roll(u, 1)` corresponds to S_+U and S_-U respectively, we can directly implement the upwind scheme with just one line of code.

2.2.2 Crank–Nicolson

The Crank–Nicolson scheme in operator form is given by:

$$U^{j+1} = (I + \frac{c\Delta t}{2}D_0)^{-1}(I - \frac{c\Delta t}{2}D_0)U^j.$$

The component-wise form of the Crank–Nicolson scheme is given by:

$$\frac{u_i^{j+1} - u_i^j}{\Delta t} + c \frac{1}{2} \left(\frac{u_{i+1}^{j+1} - u_{i-1}^{j+1}}{2\Delta x} + \frac{u_{i+1}^j - u_{i-1}^j}{2\Delta x} \right) = 0,$$

equivalently,

$$u_i^{j+1} + \frac{\lambda}{4}(u_{i+1}^{j+1} - u_{i-1}^{j+1}) = u_i^j - \frac{\lambda}{4}(u_{i+1}^j - u_{i-1}^j) \quad \text{where } \lambda = \frac{c\Delta t}{\Delta x}$$

However, the Crank–Nicolson scheme is an implicit method, which means that we need to solve a linear system of equations at each time step to compute the next solution. This is because the next solution U^{j+1} appears on both sides of the equation, and we cannot directly compute it from the current solution U^j . To do so, we would have to solve a linear system. By manipulating the operator form,

we can write the Crank–Nicolson scheme as a matrix equation $AU^{j+1} = BU^j$. We can then use the [np.linalg.solve\(\)](#) function to solve the linear system efficiently.

In our code, we have $n = \text{len}(u)$, which means our vector $U^j \in \mathbb{R}^n$ has entries

$$U^j = (u_0^j, u_1^j, \dots, u_{n-1}^j)^T$$

We then define D_0 as an $n \times n$ zero matrix, and fill in the entries according to the definition of D_0 . Specifically, we have $D_0[i, (i+1) \bmod n] = 1/(2dx)$ and $D_0[i, (i-1) \bmod n] = -1/(2dx)$ for $i = 0, 1, \dots, n-1$. We then compute the matrices A and B as follows: $A = I + \frac{c\Delta t}{2}D_0$ and $B = I - \frac{c\Delta t}{2}D_0$, where I is the identity matrix. Finally, we use [np.linalg.solve\(A, B @ u\)](#) to compute the next solution U^{j+1} .

2.2.3 Lax-Wendroff

The Lax-Wendroff scheme in operator form is given by:

$$U^{j+1} = \left(I - c\Delta t D_0 + \frac{c^2\Delta t^2}{2} D_{xx} \right) U^j$$

where D_0 and D_{xx} are defined as follows:

$$(D_0 U)_i = \frac{u_{i+1} - u_{i-1}}{2\Delta x}, \quad (D_{xx} U)_i = \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}$$

The component-wise form of the Lax-Wendroff scheme is given by:

$$u_i^{j+1} = u_i^j - \frac{v}{2}(u_{i+1}^j - u_{i-1}^j) + \frac{v^2}{2}(u_{i+1}^j - 2u_i^j + u_{i-1}^j)$$

The Lax-Wendroff scheme is very straightforward to implement, as we can directly use the definitions of D_0 and D_{xx} to compute the next solution.

2.2.4 Leapfrog

The Leapfrog scheme in operator form is given by:

$$U^{j+1} = U^{j-1} - c\Delta t D_0 U^j$$

The component-wise form of the Leapfrog scheme is given by:

$$u_i^{j+1} = u_i^{j-1} - \frac{v}{2}(u_{i+1}^j - u_{i-1}^j)$$

The Leapfrog scheme is also very straightforward to implement, as we can directly use the definition of D_0 to compute the next solution. However, it is important to note that the Leapfrog scheme is a two-step (multistep) method, which means that we need to compute the solution at two previous time steps to compute the next solution.

2.2.5 FTCDS

The FTCDS scheme in operator form is given by:

$$U^{j+1} = (I - c\Delta t D_0) U^j$$

The component-wise form of the FTCDS scheme is given by:

$$u_i^{j+1} = u_i^j - \frac{v}{2}(u_{i+1}^j - u_{i-1}^j)$$

Again, the FTCDS scheme exactly matches our code.

2.2.6 Lax-Friedrichs

The Lax-Friedrichs scheme in operator form is given by:

$$U^{j+1} = \left(\frac{1}{2}(S_+ + S_-) - c\Delta t D_0 \right) U^j$$

where S_+ and S_- are the shift operators defined as follows:

$$(S_+ U)_i = U_{i+1}, \quad (S_- U)_i = U_{i-1}$$

The component-wise form of the Lax-Friedrichs scheme is given by:

$$u_i^{j+1} = \frac{1}{2}(u_{i+1}^j + u_{i-1}^j) - \frac{v}{2}(u_{i+1}^j - u_{i-1}^j)$$

The Lax-Friedrichs scheme is easy to implement as well, as we can directly use the definitions of S_+ , S_- , and D_0 to compute the next solution.

2.2.7 RK2CDS

The RK2CDS scheme in operator form is given by:

$$U^{j+1} = \left(I - c\Delta t D_0 + \frac{c^2 \Delta t^2}{2} D_0^2 \right) U^j$$

where

$$(D_0 U^j)_i = \frac{u_{i+1}^j - u_{i-1}^j}{2\Delta x}, \quad (D_0^2 U^j)_i = D_0(D_0 U^j)_i = \frac{u_{i+2}^j - 2u_i^j + u_{i-2}^j}{4\Delta x^2}.$$

The component-wise form of the RK2CDS scheme is given by:

$$u_i^{j+1} = u_i^j - \frac{v}{2}(u_{i+1}^j - u_{i-1}^j) + \frac{v^2}{4}(u_{i+2}^j - 2u_i^j + u_{i-2}^j)$$

2.2.8 RK4CDS

The RK4CDS scheme in operator form is given by:

$$U^{j+1} = \left(I + (-c\Delta t D_4) + \frac{(-c\Delta t D_4)^2}{2} + \frac{(-c\Delta t D_4)^3}{6} + \frac{(-c\Delta t D_4)^4}{24} \right) U^j$$

According to lecture note, the RK4CDS scheme is derived by applying the classical fourth-order Runge-Kutta method to the system of ODEs obtained from the spatial semi-discretization of the PDE using central differences. The operator D_4 is defined as follows:

$$D_4 = \frac{-D_+^2 + 8D_+ - 8D_- + D_-^2}{12\Delta x}$$

which corresponds to

$$(D_4 U)_i = \frac{-U_{i+2} + 8U_{i+1} - 8U_{i-1} + U_{i-2}}{12\Delta x}$$

and that is exactly how we implement the RK4CDS scheme in our code using `np.roll()`.

2.3 conv_test.py

The `conv_test.py` file contains the implementation of the functions that are used to compute the order of convergence and stability of our numerical schemes. The main function in this file is `convergence_test()`, which takes in the errors at different grid resolutions and computes the order of convergence using a log-log plot. The function also returns the computed order of convergence, which can be used to verify the theoretical order of convergence of our numerical schemes.

Recall that we are solving the linear advection equation $u_t + cu_x = 0$ with periodic boundary conditions. The exact solution to this equation can be expressed as $u(x, t) = g(x - ct)$, where g is the initial condition i.e. $u(x, 0) = g(x)$. This means that the solution at any time t can be obtained by shifting the initial condition by an amount ct . Therefore, we can compute the exact solution at any time t by applying a shift to the initial condition, which is what we do in our code using the `exact_sol()` function. After spatial discretization, our data becomes a vector

$$U^j = (u_1^j, u_2^j, \dots, u_{n_x-1}^j)^T \in \mathbb{R}^n$$

where each entry u_i^j approximates $u(x_i, t_j)$. Note that for any one-step scheme, it corresponds exactly to our Python “scheme”(u, c, dt, dx), which is a map Φ such that

$$\Phi_{\Delta t, \Delta x}(U^j) = U^{j+1}$$

That’s also the reason why we have a specific function for the leapfrog scheme, since it is a two-step method and we need to compute U^1 using a starter one-step method before we can apply the leapfrog scheme iteratively. More details of the convergence test will be explained in 2.3.3.

2.3.1 adv(u0, scheme, c, dx, dt, T)

The `adv()` function is used to evolve the solution of the linear advection equation from time $t = 0$ to time $t = T$ using a given numerical scheme. A special feature it has is to avoid the situation that we compare U^{n_t} to somewhere “almost T” - so it ensures that we land exactly on time T . It takes in the initial condition u_0 , the scheme to be used, the wave speed c , the spatial step dx , the time step dt , and the final time T . The function iteratively applies the scheme to compute the solution at each time step until it reaches time T using the for loop.

We first have `u = u0.copy()` to avoid modifying the original initial condition. We then compute the number of time steps needed to reach time T as $n_t = \lceil \frac{T}{dt} \rceil$, and compute the effective time step $dt_{\text{eff}} = \frac{T}{n_t}$. Note that we are given a time step dt defined from CFL condition, but $\frac{T}{dt}$ may not be an integer. Hence, using `int(ceil(T/dt))` would pick the smallest integer n_t such that

$$n_t \cdot dt \geq T$$

This ensures that we take enough steps to reach time T .

2.3.2 adv_lf(u0, starter_scheme, leapfrog_scheme, c, dx, dt, T)

The idea of this function is not that different from `adv()`. The only significant difference is that the Leapfrog scheme is a two-step (multistep) method i.e.

$$U^{j+1} = \Phi_{\Delta t, \Delta x}(U^j, U^{j-1})$$

so we cannot reuse our `adv()` for the Leapfrog scheme. Though, it can be simply resolved by computing the solution at the first time step using a starter one-step method (e.g. Lax-Wendroff) before we

can apply the Leapfrog scheme iteratively. The function takes in the initial condition u_0 , the starter scheme, the leapfrog scheme, the wave speed c , the spatial step dx , the time step dt , and the final time T . The function first computes the solution at the first time step using the starter scheme, and then iteratively applies the leapfrog scheme to compute the solution at subsequent time steps until it reaches time T .

The `if np == 1` precisely checks if we are at the first time step, and if so, it uses the starter scheme to compute U^1 from U^0 . It is trivially the reason why our range is `range(1, nt)`. For subsequent time steps, it uses the leapfrog scheme to compute U^{j+1} from U^j and U^{j-1} , due to the two-step nature of the leapfrog method, our `u_prev` and `u_curr` are updated in each iteration to store the solutions at the previous two time steps, which are needed to compute the next solution using the leapfrog scheme.

2.3.3 `convergence_test(scheme_name, scheme_func, g, c, a, b, T, CFL, nx_list, is_leapfrog=False, leapfrog_starter=None)`

`convergence_test()` is the main function that performs the convergence test for a given numerical scheme. It is very important, but it is very easy to understand. The function takes in the name of the scheme - though it is not used directly within this function, it would be used in `run_conv.py` when we illustrate the results. `nxlist` is the list of grid spacings we want to test. Relating it to our `scheme_table` in `run_conv.py`, `is_leapfrog` is a boolean flag that indicates whether the scheme being tested is the leapfrog scheme, which requires a starter scheme to compute the solution at the first time step. If we are indeed using the leapfrog scheme, our manual choice would be using `schemes.lax_wendroff`. Other parameters are easy to understand.

The function iterates over the list of grid spacings: with each different grid spacing nx , it calculates xs , dx correspondingly and hence dt which correlates to dx by our supposition. It then computes the numerical solution using either the `adv()` function or the `adv_lf()` function depending on whether we are testing the leapfrog scheme, computes the exact solution at time T , and then computes the error between the numerical solution and the exact solution.

Finally, it uses the errors computed for different grid spacings to estimate the order of convergence of the numerical scheme. Note that we have

$$E(\Delta x) \approx C(\Delta x)^p$$

(note that this is our mathematical cornerstone of the convergence test theoretically)

Then by taking two successive Δx_{k-1} , Δx_k and their corresponding errors E_{k-1} , E_k , we can derive that

$$\frac{E_{k-1}}{E_k} \approx \frac{C(\Delta x_{k-1})^p}{C(\Delta x_k)^p} = \left(\frac{\Delta x_{k-1}}{\Delta x_k} \right)^p$$

hence, we just take the logs to compute p :

$$p \approx \frac{\log(E_{k-1}/E_k)}{\log(\Delta x_{k-1}/\Delta x_k)}$$

which is essentially what the last block of code in `convergence_test()` does.

As a result, it returns the list of grid spacings, the list of effective time steps, the list of errors, and the list of estimated orders of convergence.

2.4 stab_test.py

2.4.1 stability(name, leapfrog_scheme, g, c, a, b, nx, CFL, Ts, is_leapfrog=False, starter_scheme=None)

The `stability()` function is used to test the stability of a given numerical scheme. It takes in the name of the scheme, the leapfrog scheme (if applicable), the initial condition, the wave speed, the spatial domain, the number of grid points, the CFL number, the list of final times to test, and a boolean flag indicating whether the scheme being tested is the leapfrog scheme. The function iterates over the list of final times, computes the numerical solution at each final time using either the `adv()` function or the `adv_lf()` function depending on whether we are testing the leapfrog scheme, and then computes the ratio of the maximum absolute value of the numerical solution to the maximum absolute value of the initial condition. This ratio is used as an indicator of stability, as a stable scheme should not produce solutions that grow unboundedly over time. The function returns a list of final times and their corresponding stability ratios, which can be used to analyze the stability of the numerical scheme.

We fix CFL and nx for all the schemes. The first two lines of code compute the grid points and the spacing, with specifying the initial condition. With this initial data, we compute its max using `np.max(np.abs(u0))` to be used as the denominator in our stability ratio.

Note that since we are varying our final time T while keeping the time step dt fixed (as it is determined by the CFL condition), we are essentially increasing the number of time steps n_t as we increase T . This allows us to observe how the numerical solution evolves over a longer time period, and whether it remains stable or grows unboundedly. The reason why “effCFL” is of particular interest is that when our final time T varies, the corresponding nt would also vary. Just like in `adv()`, `adv_lf()`, we use the if-statement to distinguish the leapfrog scheme from the other schemes.

Lastly, it is worth noting that since we are checking stability, then it is naturally reasonable to consider the bad cases where it goes out of bound as T increases, especially when we have a division. That’s why I introduced the if-statement. The `not np.all(np.isfinite(uT))` checks if all the entries of the numerical solution at time T are finite, which is a necessary condition for stability. If any entry is infinite or NaN, it indicates that the solution has blown up, and we can immediately conclude that the scheme is unstable for that particular final time T . In such cases, we set the stability ratio to infinity to reflect the instability of the scheme. The second condition `not np.isfinite(amp0)` checks if the maximum absolute value of the initial condition is finite. If it is not finite, it would lead to a division by infinity, which would not provide meaningful information about stability. In such cases, we also set the stability ratio to infinity. The last condition `amp0 == 0` checks if the maximum absolute value of the initial condition is when it is identically zero, which would lead to a division by zero when computing the stability ratio, for which we still set it to infinity to reflect the fact that we cannot determine stability in this case. If we have any other conditions, for which they should be valid, we proceed to compute the stability ratio as the maximum absolute value of the numerical solution at time T divided by the maximum absolute value of the initial condition, which gives us a measure of how much the solution has grown relative to the initial condition.

2.5 run_cfl.py

Note that I do not have a separated .py file i.e. `cfl_test.py` like the previous two experiments: I had a hard time trying to import the main testing function into `run_cfl.py` from my now-deleted `cfl_test.py`, so I just implemented the CFL test directly in `run_cfl.py` for simplicity - which worked. With that being said, `check_cfl()` is highly alike to `stability()`, so I will skip the explanation of the code here.

Chapter 3: Results and Analysis

We present the results of our numerical experiments in the form of tables and plots. The tables show the errors and estimated orders of convergence for each scheme at different grid resolutions, while the plots show the log-log relationship between the error and the grid spacing, which allows us to visually verify the order of convergence.

3.1 Order of Convergence

The order of convergence for each scheme is estimated using the errors computed at different grid resolutions. The results are summarized in the following tables and plots.

upwind				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.02095402	-
100	0.062831853	0.043478261	0.0096313174	1.121
200	0.031415927	0.02173913	0.0048268613	0.997
400	0.015707963	0.010989011	0.0023567125	1.034
800	0.0078539816	0.0054945055	0.0011790447	0.999
ftcds				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.042467919	-
100	0.062831853	0.043478261	0.021965515	0.951
200	0.031415927	0.02173913	0.010927439	1.007
400	0.015707963	0.010989011	0.0055094521	0.988
800	0.0078539816	0.0054945055	0.56946546	-6.692
lax_friedrichs				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.051734648	-
100	0.062831853	0.043478261	0.023386503	1.145
200	0.031415927	0.02173913	0.011761215	0.992
400	0.015707963	0.010989011	0.0057158156	1.041
800	0.0078539816	0.0054945055	0.0028619869	0.998
lax_wendroff				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.0014730693	-
100	0.062831853	0.043478261	0.0003428321	2.103
200	0.031415927	0.02173913	8.5723393e-05	2.000
400	0.015707963	0.010989011	2.099665e-05	2.030
800	0.0078539816	0.0054945055	5.2492203e-06	2.000

Figure 3.1: Convergence1

rk2cds				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.0014856812	-
100	0.062831853	0.043478261	0.00034373799	2.112
200	0.031415927	0.02173913	8.578e-05	2.003
400	0.015707963	0.010989011	2.1000319e-05	2.030
800	0.0078539816	0.0054945055	5.2494496e-06	2.000
rk4cd4				
nx	dx	dt	error	order
50	0.12566371	0.083333333	8.6975397e-06	-
100	0.062831853	0.043478261	5.4903074e-07	3.986
200	0.031415927	0.02173913	3.4326762e-08	3.999
400	0.015707963	0.010989011	2.1508147e-09	3.996
800	0.0078539816	0.0054945055	1.3443048e-10	4.000
crank_nicolson				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.0032033717	-
100	0.062831853	0.043478261	0.00081501848	1.975
200	0.031415927	0.02173913	0.00020384555	1.999
400	0.015707963	0.010989011	5.1184618e-05	1.994
800	0.0078539816	0.0054945055	1.2796516e-05	2.000
leapfrog				
nx	dx	dt	error	order
50	0.12566371	0.083333333	0.00148145	-
100	0.062831853	0.043478261	0.00034291646	2.111
200	0.031415927	0.02173913	8.5756265e-05	2.000
400	0.015707963	0.010989011	2.0996975e-05	2.030
800	0.0078539816	0.0054945055	5.2493486e-06	2.000

Figure 3.2: Convergence2

From the tables, we can see that the estimated orders of convergence for each scheme are consistent with our expectations based on the theoretical analysis. The upwind, FTCDS, and Lax-Friedrichs schemes show a first-order convergence, while the Leapfrog, Crank–Nicolson, RK2CDS, and Lax-Wendroff schemes show a second-order convergence. Lastly, the RK4CDS scheme shows a fourth-order convergence.

The log-log plots of error versus grid spacing also show straight lines with slopes corresponding to the estimated orders of convergence, which visually confirms our results. The upwind, FTCDS, and Lax-Friedrichs schemes have slopes close to 1, while the Leapfrog, Crank–Nicolson, RK2CDS, and Lax-Wendroff schemes have slopes close to 2, and the RK4CDS scheme has a slope close to 4. Though, it can be clearly seen that the FTCDS scheme blows up when $\Delta x \rightarrow 0$, which indicates that it is not stable under the given CFL condition.

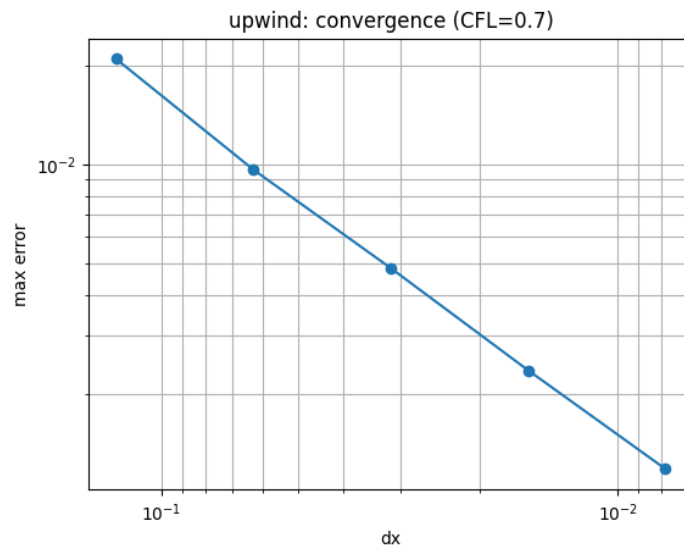


Figure 3.3: Upwind scheme convergence

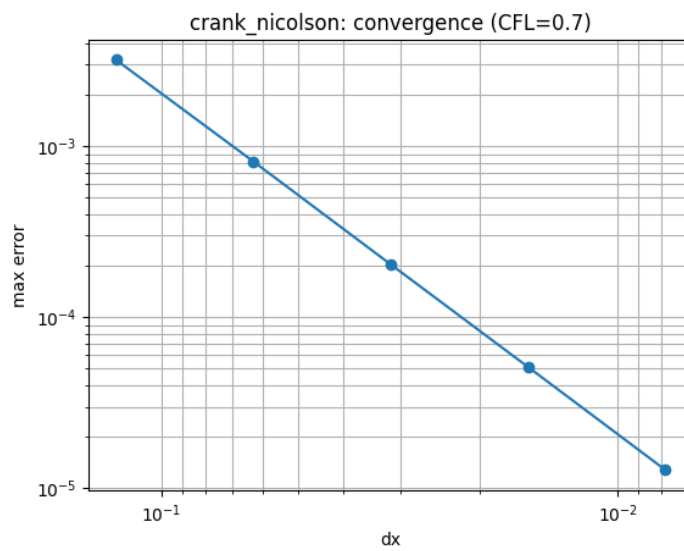


Figure 3.4: Crank–Nicolson scheme convergence

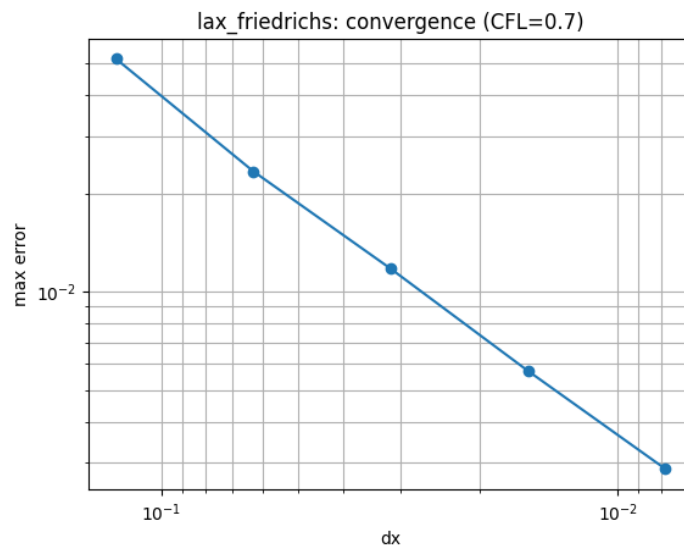


Figure 3.5: Lax-Friedrichs scheme convergence

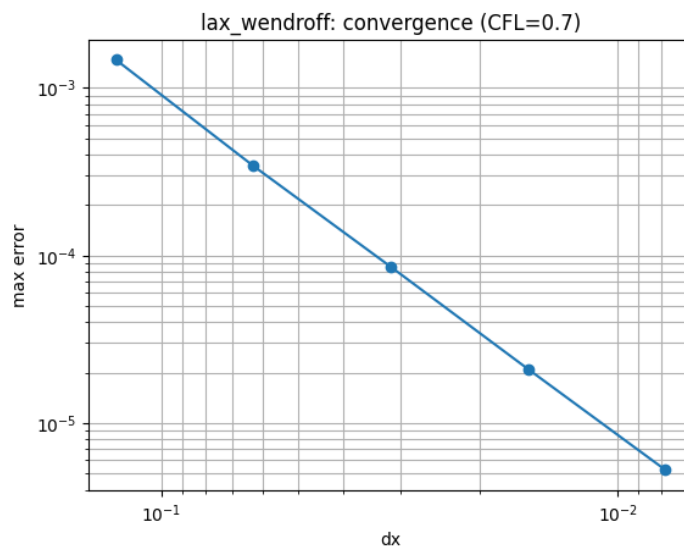


Figure 3.6: Lax-Wendroff scheme convergence

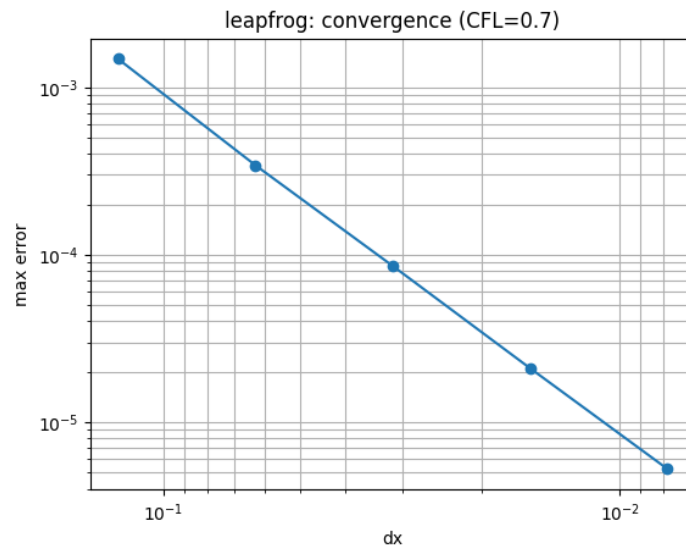


Figure 3.7: Leapfrog scheme convergence

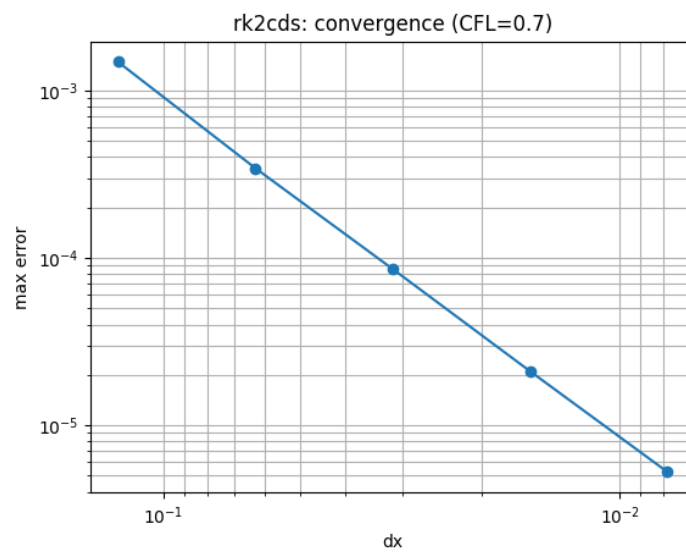


Figure 3.8: RK2CDS scheme convergence

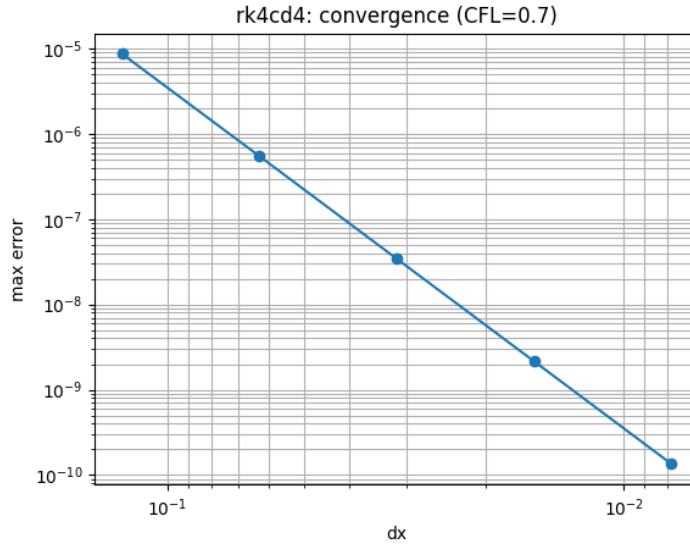


Figure 3.9: RK4CDS scheme convergence

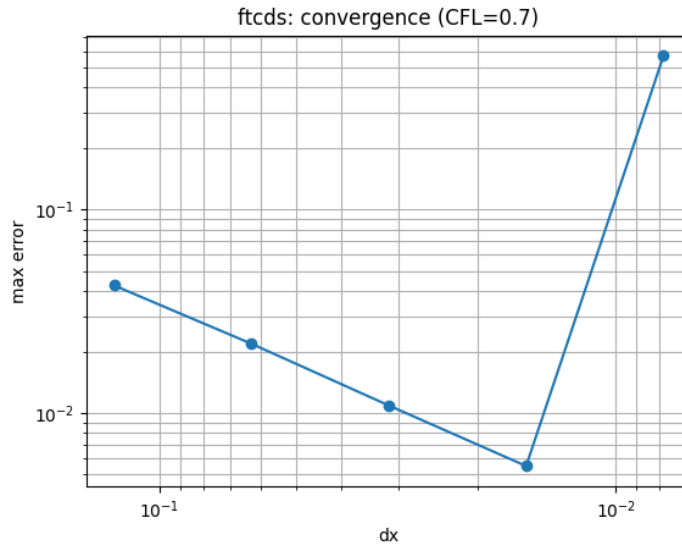


Figure 3.10: FTCDS scheme convergence

As can be seen, RK4CDS has the smallest error among all the schemes (i.e. scale goes all the way to 10^{-10}), which is consistent with its fourth-order convergence since all the other schemes are of lower order. The FTCDS scheme, on the other hand, uncontrollably blows up as Δx decreases, which indicates that it is not stable under the given CFL condition. In order to further investigate, we test on $\text{CFL} \in \{0.05, 0.1, 0.2, 0.5, 0.9\}$. The result is quite interesting in the sense that up to $\text{CFL} = 0.5$, the FTCDS scheme is stable and converges with a first-order convergence, but when we increase the CFL number to 0.6, the scheme starts to blow up as Δx decreases, which indicates that the stability of the FTCDS scheme is very sensitive to the choice of the CFL number, and it is not stable when the CFL number exceeds 0.5.

ftcds				
nx	dx	dt	error	order
50	0.12566371	0.00625	0.0040864176	-
100	0.062831853	0.0031347962	0.0017005436	1.265
200	0.031415927	0.0015698587	0.00080220839	1.084
400	0.015707963	0.00078492936	0.00039467991	1.023
800	0.0078539816	0.00039261877	0.00019659635	1.005

Figure 3.11: FTCDS scheme convergence CFL = 0.05

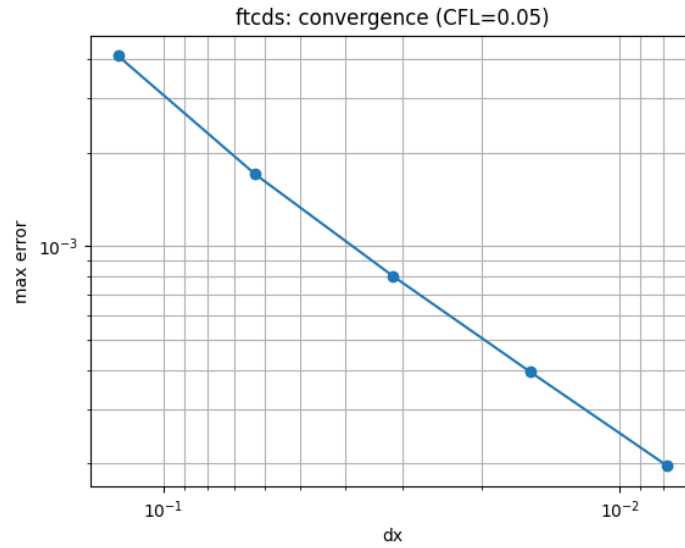


Figure 3.12: FTCDS graph convergence CFL = 0.05

ftcds				
nx	dx	dt	error	order
50	0.12566371	0.0125	0.006791431	-
100	0.062831853	0.00625	0.0031971011	1.087
200	0.031415927	0.0031347962	0.0015770651	1.020
400	0.015707963	0.0015698587	0.00078629234	1.004
800	0.0078539816	0.00078492936	0.0003926736	1.002

Figure 3.13: FTCDS scheme convergence CFL = 0.1

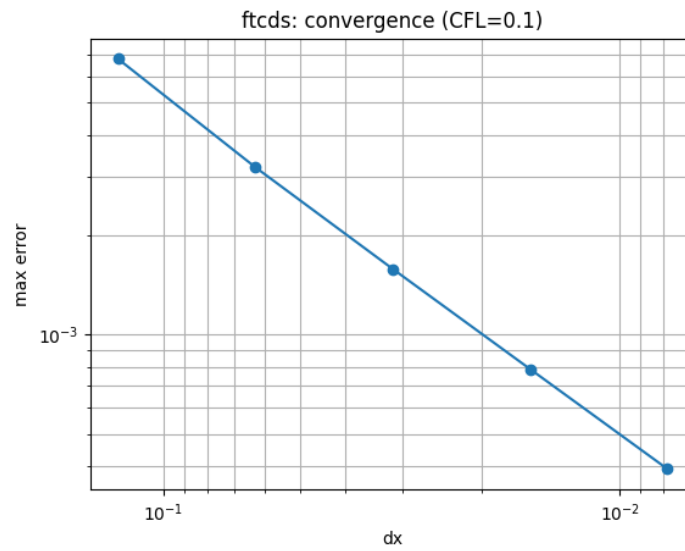


Figure 3.14: FTCDS graph convergence CFL = 0.1

ftcds				
nx	dx	dt	error	order
50	0.12566371	0.025	0.012829555	-
100	0.062831853	0.0125	0.0063011699	1.026
200	0.031415927	0.00625	0.0031338421	1.008
400	0.015707963	0.0031347962	0.0015691195	0.998
800	0.0078539816	0.0015698587	0.00078529898	0.999

Figure 3.15: FTCDS scheme convergence CFL = 0.2

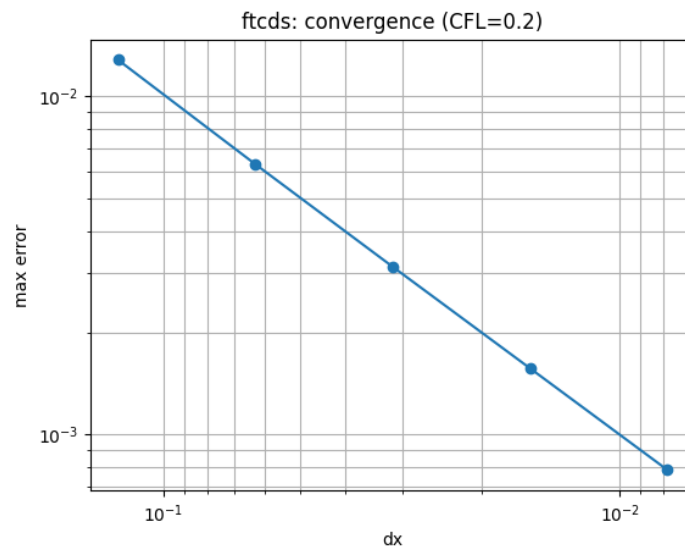


Figure 3.16: FTCDS graph convergence CFL = 0.2

ftcds				
nx	dx	dt	error	order
50	0.12566371	0.0625	0.031762258	-
100	0.062831853	0.03125	0.015750269	1.012
200	0.031415927	0.015625	0.0078434299	1.006
400	0.015707963	0.0078125	0.0039139319	1.003
800	0.0078539816	0.0039215686	0.0021505487	0.864

Figure 3.17: FTCDS scheme convergence CFL = 0.5

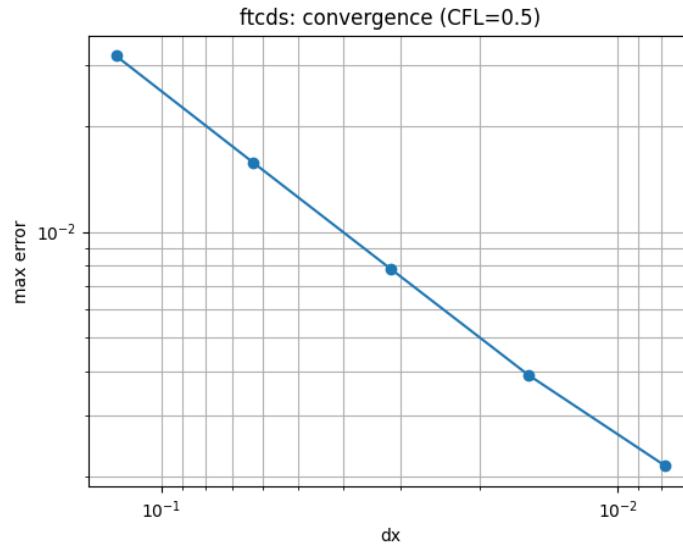


Figure 3.18: FTCDS graph convergence CFL = 0.5

ftcds				
nx	dx	dt	error	order
50	0.12566371	0.071428571	0.036338246	-
100	0.062831853	0.037037037	0.018687189	0.959
200	0.031415927	0.018518519	0.0093017992	1.006
400	0.015707963	0.0093457944	0.0046837714	0.990
800	0.0078539816	0.0046948357	0.016252813	-1.795

Figure 3.19: FTCDS scheme convergence CFL = 0.6

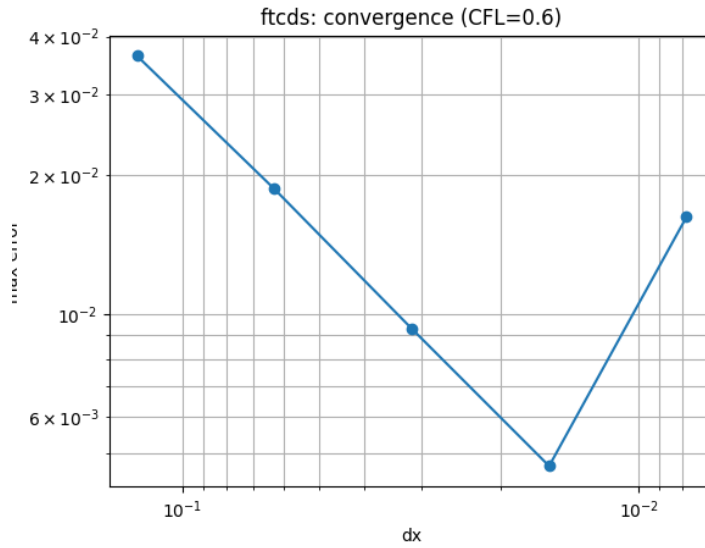


Figure 3.20: FTCDS graph convergence CFL = 0.6

ftcds				
nx	dx	dt	error	order
50	0.12566371	0.11111111	0.05687992	-
100	0.062831853	0.055555556	0.028137526	1.015
200	0.031415927	0.027777778	0.01398217	1.009
400	0.015707963	0.014084507	0.0070667567	0.984
800	0.0078539816	0.0070422535	127.79743	-14.142

Figure 3.21: FTCDS scheme convergence CFL = 0.9

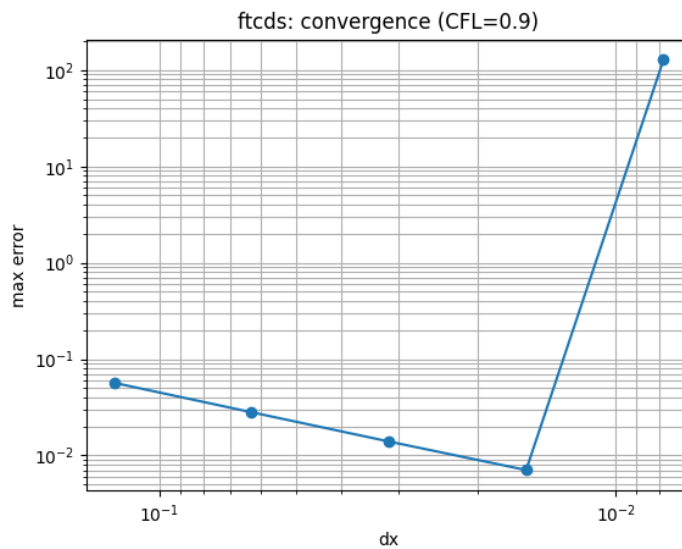


Figure 3.22: FTCDS graph convergence CFL = 0.9

This finishes our analysis of the order of convergence for the different numerical schemes. The results are consistent with the theoretical expectations, and they highlight the importance of choosing an appropriate scheme and CFL number to ensure stability and accuracy in numerical simulations.

3.2 Stability

The stability of each scheme is investigated by picking a fixed CFL and grid spacing (e.g. CFL = 0.6 and $\Delta x = 200$) so it will not blow up due to the choice of CFL and grid spacing, and then we test the schemes with different final time T : for several T , we compute $u(x, T)$ using each scheme and compare it to $u(x, 0)$ to see if the solution blows up or not.

The idea is that we define the growth factor as $G(T) = \frac{\|u(x, T)\|}{\|u(x, 0)\|}$, which measures how much the solution grows over time. If the growth factor is around 1 or decreases as T increases, then we can conclude that the scheme is stable under the given CFL condition. On the other hand, if the growth factor grows out of bound with T , then we can conclude that the scheme is unstable under the given CFL condition.

```
upwind
T      nt      effCFL      growth
1.0    54      0.589      0.9935718061
5.0    266     0.598      0.9689439790
10.0   531     0.599      0.9390186349
50.0   2653    0.600      0.7303402048
100.0  5306    0.600      0.5333968148

c:\Users\shake\OneDrive\Desktop\All Files\VSCode\DI0 Project1\schemes.py:37: RuntimeWarning: overflow encountered in subtract
  return u - 0.5 * nu * (np.roll(u, -1) - np.roll(u, 1))
c:\Users\shake\OneDrive\Desktop\All Files\VSCode\DI0 Project1\schemes.py:37: RuntimeWarning: invalid value encountered in multiply
  return u - 0.5 * nu * (np.roll(u, -1) - np.roll(u, 1))

ftcds
T      nt      effCFL      growth
1.0    54      0.589      1.0092975837
5.0    266     0.598      33.0214381644
10.0   531     0.599      10287924880136026112.0000000000
50.0   2653    0.600      28304677551386834358470891615139441815734127482719494639479844354876136529649844681940497531671657992834238212199184879227154843934085266620851691369327681863680.0000000000
100.0  5306    0.600      inf
```

Figure 3.23: Schemes Stability 1

```
lax_friedrichs
T      nt      effCFL      growth
1.0    54      0.589      0.9827617576
5.0    266     0.598      0.9191809942
10.0   531     0.599      0.8454604590
50.0   2653    0.600      0.4325630007
100.0  5306    0.600      0.1871107495

lax_wendroff
T      nt      effCFL      growth
1.0    54      0.589      0.9999985094
5.0    266     0.598      0.9999925572
10.0   531     0.599      0.9999851179
50.0   2653    0.600      0.9999255985
100.0  5306    0.600      0.9998512025

rk2cds
T      nt      effCFL      growth
1.0    54      0.589      1.0000007933
5.0    266     0.598      1.0000041482
10.0   531     0.599      1.0000083434
50.0   2653    0.600      214.3672620715
100.0  5306    0.600      313114583325401546752.0000000000

rk4cd4
T      nt      effCFL      growth
1.0    54      0.589      1.0000000000
5.0    266     0.598      0.9999999999
10.0   531     0.599      0.9999999998
50.0   2653    0.600      0.9999999992
100.0  5306    0.600      0.9999999983

crank_nicolson
T      nt      effCFL      growth
1.0    54      0.589      1.0000000000
5.0    266     0.598      1.0000000000
10.0   531     0.599      1.0000000000
50.0   2653    0.600      1.0000000000
100.0  5306    0.600      1.0000000000

leapfrog
T      nt      effCFL      growth
1.0    54      0.589      0.9999999805
5.0    266     0.598      0.9999999743
10.0   531     0.599      0.9999999802
50.0   2653    0.600      0.9999999740
100.0  5306    0.600      0.9999999925
```

Figure 3.24: Schemes Stability 2

From the tables, we can see that the upwind, Crank–Nicolson, Lax-Wendroff, Lax-Friedrichs, Leapfrog, and RK4CDS schemes are stable under the given CFL condition, as their growth factors remain around 1 or decrease as T increases. On the other hand, the FTCDS scheme is extremely unstable under the given CFL condition, as its growth factor grows out of bound with T . Note that we have a “runtime-warning” for the FTCDS scheme when T is large, which indicates that the solution has blown up. Another scheme that blows up is the RK2CDS scheme, but slightly better than the FTCDS scheme.

It is worth mentioning that the table of Crank-Nicolson does not show any changes in growth. This is due to the fact that I set the display for growth to be of 10 decimal places - it definitely changed, it is just too small to be detected with 10 d.p.. Since we are only investigating the stability of the schemes, this directly shows that while T increases, Crank-Nicolson remains incredibly stable. The best ones are Crank-Nicolson, RK4CDS, Leapfrog and Lax-Wendroff, while the remaining two schemes are still stable but decreases as T increases.

We provide the corresponding graphs to further illustrate the stability/instability of the schemes. (Note the scale of the y-axis of each). We claimed Leapfrog to be stable, but the graph fluctuates a little bit - which is fine since it is fluctuating in a small interval, which does not directly signal instability. To see the stability of the Leapfrog scheme, I have modified T to be specifically up to 10000, for which the result is still within expectation - it still fluctuates a lot, but in a reasonable small interval and does not blow up. (Check the last graph).

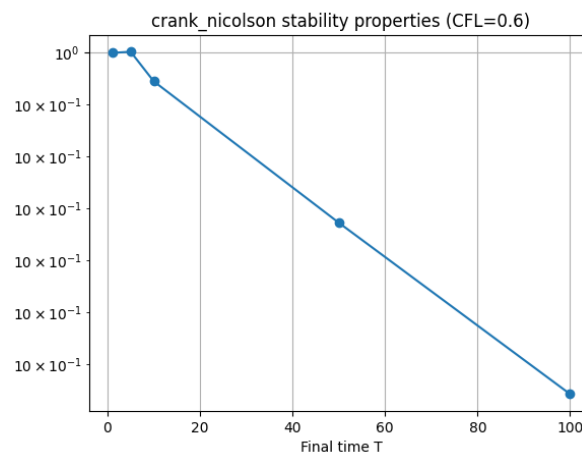


Figure 3.25: Crank-Nicolson Stability

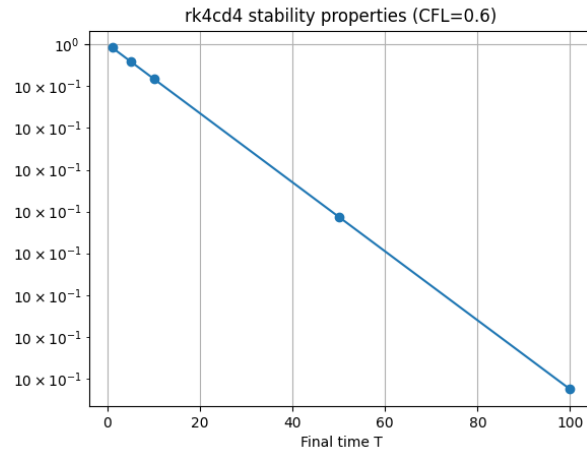


Figure 3.26: RK4CDS Stability

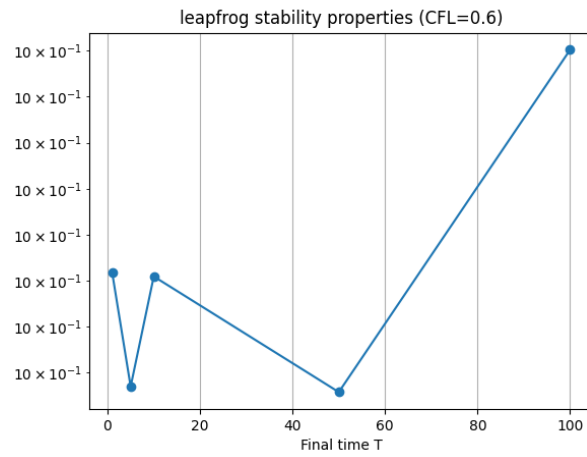


Figure 3.27: Leapfrog Stability

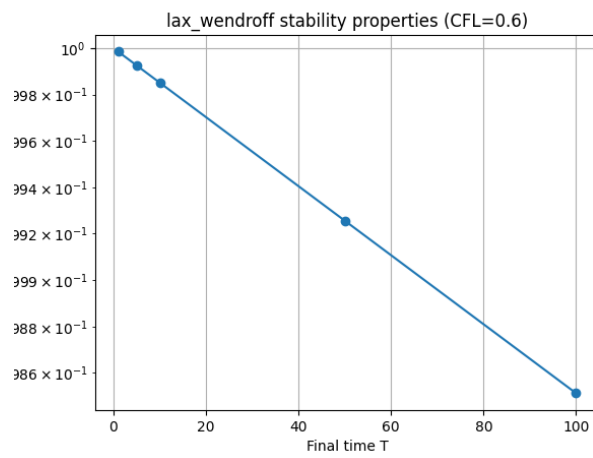


Figure 3.28: Lax-Wendroff Stability

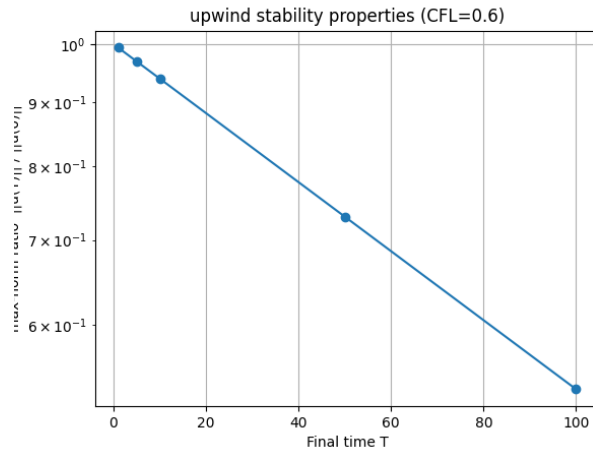


Figure 3.29: Upwind Stability

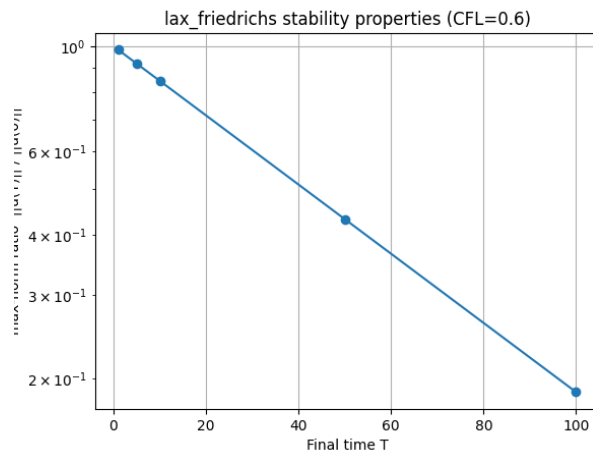


Figure 3.30: Lax-Friedrichs Stability

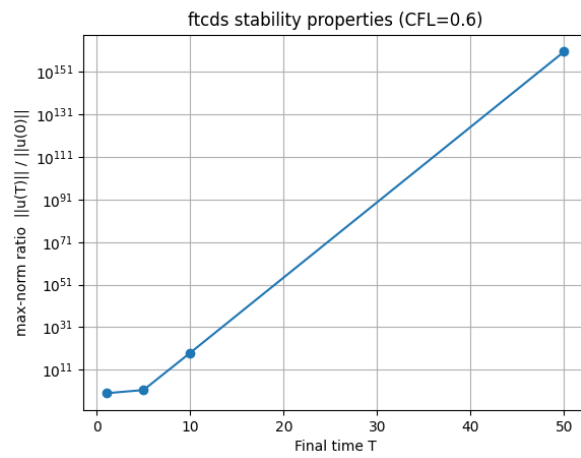


Figure 3.31: FTCDs Stability

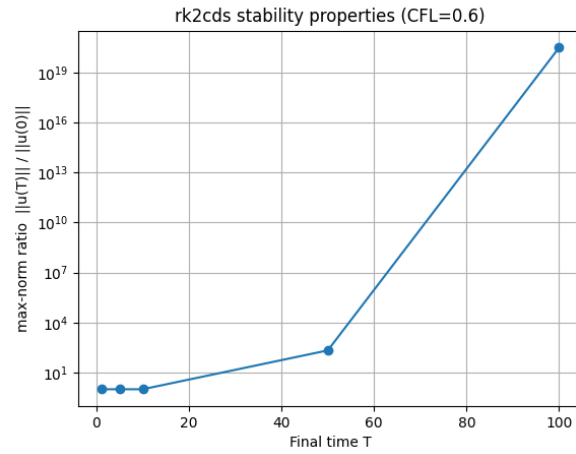


Figure 3.32: RK2CDS Stability

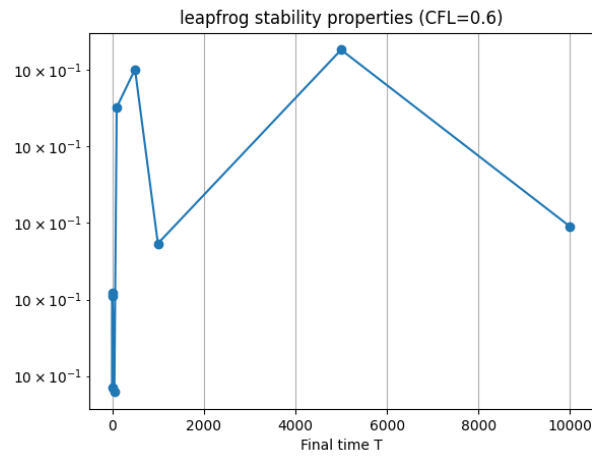


Figure 3.33: Leapfrog Stability (T = 10000)

leapfrog			
T	nt	effCFL	growth
1.0	54	0.5894627522	0.9999999805
5.0	266	0.5983268537	0.9999999743
10.0	531	0.5994536463	0.9999999802
50.0	2653	0.5999055526	0.9999999740
100.0	5306	0.5999055526	0.9999999925
500.0	26526	0.5999960156	0.9999999950
1000.0	53052	0.5999960156	0.9999999837
5000.0	265259	0.5999982775	0.9999999963
10000.0	530517	0.5999994085	0.9999999848

Figure 3.34: Leapfrog Stability Table (T = 10000)

This finishes our discussion on the stability of the different numerical schemes.

3.3 CFL Condition

The experiment designed to investigate how CFL conditions affect the stability of the schemes will be highly identical to the stability test we have just done, except that we will be varying the CFL number instead of the final time T - since we are still checking the stability of the schemes.

However, instead of solely using different CFLs, I will also vary the final time T . The idea is that if CFL truly affects the stability of the schemes, then we should see the same pattern as CFL changes, with each final time T . This will further validates our conclusion since the potential outcomes with different final times T are considered.

```

upwind
T = 5
targetCFL    nt      effCFL      growth
0.200        796      0.200      0.9390973877
0.400        398      0.400      0.9539590944
0.800        199      0.800      0.9043973079
1.000        160      0.995      0.9995853027
5.000         32      4.974      279230149849122.0625000000
10.000        16      9.947      30174.3381160367

T = 10
targetCFL    nt      effCFL      growth
0.200        1592     0.200      0.8819030936
0.400         796     0.400      0.9100379539
0.800         398     0.800      0.9690300597
1.000         319     0.998      0.9996602645
5.000          64     4.974      517478448277025808080662740953156286650253312.0000000000
10.000         32     9.947      6746797781105925299372032.0000000000

T = 20
targetCFL    nt      effCFL      growth
0.200        3184     0.200      0.7777544953
0.400        1592     0.400      0.8281600775
0.800         796     0.800      0.9390347612
1.000         637     0.999      0.9998125101
5.000         128     4.974      24007632809757170883472215885405336379128776362128340180664418232996419049284907294197642329125896374927360.0000000000
10.000         64     9.947      305557502124308597915662027972387264921930296768552997100632145920.0000000000

```

Figure 3.35: Upwind CFL Stability

We first talk about the upwind scheme. As can be seen from the graph, with different final times T , the pattern remains the same throughout - hence the final time is not of our consideration. Up until CFL = 1, the growth factor remains around 1, which indicates that the upwind scheme is stable under the given CFL condition. However, when we further increase the CFL number, the growth factor starts to grow out of bound with T , which indicates that the upwind scheme is unstable under the given CFL condition when CFL exceeds 1.

```

ftcds
T = 5
targetCFL    nt      effCFL      growth
0.200        796      0.200      1.0158219049
0.400        398      0.400      1.0325628207
0.800        199      0.800      134681.3157680721
1.000        160      0.995      46355985.1183132008
5.000         32      4.974      5859610.9197292421
10.000        16      9.947      4.2286762715

T = 10
targetCFL    nt      effCFL      growth
0.200        1592     0.200      1.0375525306
0.400         796     0.400      2282867172.9506840706
0.800         398     0.800      206159810239873698167259136.0000000000
1.000         319     0.998      32022941816250307271564663455744.0000000000
5.000          64     4.974      122506611866081199254187016192.0000000000
10.000         32     9.947      15170922631490158.0000000000

T = 20
targetCFL    nt      effCFL      growth
0.200        3184     0.200      163485447752.0353698730
0.400        1592     0.400      83712897721814024342728301692846080.0000000000
0.800         796     0.800      1041183849115328742229510145806262523277374428860107017581911605248000.0000000000
1.000         637     0.999      22440930830693516247743372839950198010861066727233224740676472375496246936207360.0000000000
5.000         128     4.974      94121198784415403954879719411891223996340274644606426976906743402870079488.0000000000
10.000         64     9.947      859502590525175584296414432806695059267734470656.0000000000

```

Figure 3.36: FTCDS CFL Stability

The FTCDS scheme is interesting to analyze. First of all, there is certainly an interplay between the

final times T and the CFL number: we see that as T increases, the previously relatively stable growth with the small CFL numbers eventually grows out of bound. This demonstrates that the stability of the FTCDS scheme is extremely volatile, many factors can affect its stability. From our experiment, the furthest we can conclude is that the FTCDS scheme is only relatively stable when the CFL number is small and the final time T is small, but it becomes unstable as either of them increases.

lax_friedrichs			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	0.6858054163
0.400	398	0.400	0.8478833487
0.800	199	0.800	0.9652355220
1.000	160	0.995	0.9991687798
5.000	32	4.974	2878363.3121610275
10.000	16	9.947	3.9637234686
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	0.4703290690
0.400	796	0.400	0.7189061731
0.800	398	0.800	0.9316796129
1.000	319	0.998	0.9993200753
5.000	64	4.974	30806338638034786043123728384.0000000000
10.000	32	9.947	12199285218734142.0000000000
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	0.2212094332
0.400	1592	0.400	0.5168260857
0.800	796	0.800	0.8680269012
1.000	637	0.999	0.9996250359
5.000	128	4.974	6979020266892036290189565339435707607832918805754666920707106089889431552.0000000000
10.000	64	9.947	560391448138067192352132995700102008831922405376.0000000000

Figure 3.37: Lax-Friedrichs CFL Stability

The Lax-Friedrichs scheme is stable under the given CFL condition when CFL is less than or equal to 1, as the growth factor remains around 1. However, when we further increase the CFL number, the growth factor starts to grow out of bound with T unpredictably, which indicates that the Lax-Friedrichs scheme is unstable under the given CFL condition when CFL exceeds 1.

lax_wendroff			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	0.999962888
0.400	398	0.400	0.999934089
0.800	199	0.800	0.999944158
1.000	160	0.995	0.999977909
5.000	32	4.974	53643078242579306675119839026102665216.0000000000
10.000	16	9.947	49243765725130831104.0000000000
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	0.999925517
0.400	796	0.400	0.999868019
0.800	398	0.800	0.999883116
1.000	319	0.998	0.999998129
5.000	64	4.974	26747172945186574405793842206776144034180904109737340265137305552280460267990899292553216.0000000000
10.000	32	9.947	162512365927020574150278203963097825732368480724083671840.0000000000
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	0.999851234
0.400	1592	0.400	0.999739639
0.800	796	0.800	0.999776633
1.000	637	0.999	0.999999076
5.000	128	4.974	12139532547265832067775058074795975311210876291302625051042179899478041304047568678747500292123751498563062343046606279825866755854923808975680306201454726041532990205867213259820
10.000	64	9.947	2424570100229504017172431200250900893706340812085853086604619533014218602640705570699681414155096726406637081689917925317402427302.0000000000

Figure 3.38: Lax-Wendroff CFL Stability

Similar to the Lax-Friedrichs scheme, the Lax-Wendroff scheme is stable under the given CFL condition when CFL is less than or equal to 1, as the growth factor remains around 1. Solely based on the numbers computed, we see that the Lax-Wendroff scheme blows up much faster than the Lax-Friedrichs scheme.

leapfrog			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	0.9999999957
0.400	398	0.400	0.9999999850
0.800	199	0.800	0.9999999777
1.000	160	0.995	0.9999999888
5.000	32	4.974	20686289938358624.000000000
10.000	16	9.947	540540.9916855233
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	0.9999999986
0.400	796	0.400	0.9999999952
0.800	398	0.800	0.9999999917
1.000	319	0.998	0.9999999996
5.000	64	4.974	721284589588073317748954902996558457445499797504.000000000
10.000	32	9.947	214968545452515805483761664.000000000
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	0.9999999961
0.400	1592	0.400	0.9999999864
0.800	796	0.800	0.9999999766
1.000	637	0.999	1.0000000000
5.000	128	4.974	1477351030733764779568553265475846417729649294114563988771914855697553679713253749156192827233367567429042962432.000000000
10.000	64	9.947	41388757133566550437134564234981528999423917506665105645903303147520.000000000

Figure 3.39: Leapfrog CFL Stability

Again, similar to the Lax-Friedrichs and Lax-Wendroff schemes, the Leapfrog scheme is stable under the given CFL condition when CFL is less than or equal to 1, as the growth factor remains around 1.

rk2cds			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	1.0000001548
0.400	398	0.400	1.0000012384
0.800	199	0.800	1.0000099071
1.000	160	0.995	1.0000190637
5.000	32	4.974	9524764564517021696.000000000
10.000	16	9.947	280336895122.1271972656
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	1.0000003096
0.400	796	0.400	1.0000024768
0.800	398	0.800	1.0000198317
1.000	310	0.990	1.0003263107
5.000	64	4.974	46136356749114515252254531114629314208874285671859712.000000000
10.000	32	9.947	149608402201434561898694625938508873728.000000000
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	1.0000006192
0.400	1592	0.400	1.0000049535
0.800	796	0.800	5.5011882639
1.000	637	0.999	336794855183332.0625000000
5.000	128	4.974	33708005948630208977615451670944977204912065389129926642118637080900791934580607931366188853926494796796771738657228684787712.000000000
10.000	64	9.947	146293089673207902263409671295116920702675259389652931608892476454437850338096549347698147328.000000000

Figure 3.40: RK2CDS CFL Stability

rk4cds			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	1.0000000000
0.400	398	0.400	1.0000000000
0.800	199	0.800	0.9999999997
1.000	160	0.995	0.9999999990
5.000	32	4.974	748849459386296338954655973763174491836382231.000000000
10.000	16	9.947	1391368332176669211873497246521584.000000000
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	1.0000000000
0.400	796	0.400	1.0000000000
0.800	398	0.800	0.9999999993
1.000	319	0.998	0.9999999979
5.000	64	4.974	99557825893418713237525098435270349094809358669737682985715381482906340647731943953669756873433788973056.000000000
10.000	32	9.947	269878500540494986951914682501286167133861495743124159012969011092152641962861658112.000000000
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	1.0000000000
0.400	1592	0.400	1.0000000000
0.800	796	0.800	0.9999999998
1.000	637	0.999	0.9999999958
5.000	128	4.974	235554091881847040126160651993374641372649305429667641632420755322683618501414073724764343294515198268367363305769906305261533201801454400058676480175247050130771814613186388854242660
10.000	64	9.947	2624205074347139122489796419761915928819285094784131288888149192911709160382366754665755197792092117832960065346354002555743404944802392717497043255055948678172854057418267320523948032.000000000

Figure 3.41: RK4CDS CFL Stability

The RK2CDS and RK4CDS schemes are stable under the given CFL condition when CFL is less than or equal to 1, as the growth factor remains around 1. However, one thing we observe is that RK4CDS is more relatively insensitive to changes in CFL as final time increases: when $T = 20$, the growth factor starts to blow up though not exponentially with CFL numbers ≥ 0.8 ; but RK4CDS remains stable under these circumstances.

crank_nicolson			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	1.0000000000
0.400	398	0.400	1.0000000000
0.800	199	0.800	1.0000000000
1.000	160	0.995	1.0000000000
5.000	32	4.974	1.0000000000
10.000	16	9.947	1.0000000000
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	1.0000000000
0.400	796	0.400	1.0000000000
0.800	398	0.800	1.0000000000
1.000	319	0.998	1.0000000000
5.000	64	4.974	1.0000000000
10.000	32	9.947	1.0000000000
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	1.0000000000
0.400	1592	0.400	1.0000000000
0.800	796	0.800	1.0000000000
1.000	637	0.999	1.0000000000
5.000	128	4.974	1.0000000000
10.000	64	9.947	1.0000000000

Figure 3.42: Crank-Nicolson CFL Stability

Lastly, the Crank-Nicolson scheme is (unsurprisingly) stable under the given CFL condition for all the CFL numbers tested with high involatility to different final times T , as the growth factor remains around 1 for all CFL numbers and final times. This is consistent with how Crank-Nicolson is stable for the previous tests. Again, it does not show any changes because I wanted to have consistent display for all the schemes, but it is indeed changing with different CFL numbers and final times, just that the changes are too small to be detected with 10 d.p.. The actual change is illustrated in the following table where I increased the decimal places to 20.

crank_nicolson			
T = 5			
targetCFL	nt	effCFL	growth
0.200	796	0.200	0.99999999999994559907
0.400	398	0.400	0.9999999999999966693
0.800	199	0.800	0.99999999999998345768
1.000	160	0.995	1.0000000000000088818
5.000	32	4.974	0.999999999999722444
10.000	16	9.947	0.999999999999900080
T = 10			
targetCFL	nt	effCFL	growth
0.200	1592	0.200	0.99999999999988298249
0.400	796	0.400	0.999999999999900080
0.800	398	0.800	0.9999999999996480593
1.000	319	0.998	1.0000000000000022204
5.000	64	4.974	0.999999999999378275
10.000	32	9.947	0.999999999999744649
T = 20			
targetCFL	nt	effCFL	growth
0.200	3184	0.200	0.99999999999976185716
0.400	1592	0.400	0.9999999999999156231
0.800	796	0.800	0.9999999999992694732
1.000	637	0.999	0.9999999999998445688
5.000	128	4.974	0.9999999999998467892
10.000	64	9.947	0.999999999999344968

Figure 3.43: Crank-Nicolson CFL Stability Table (20 d.p.)